docloom: An Intermediate Representation for LLM-Generated Documents

A technical whitepaper describing schema-validated IR, deterministic rendering, and multi-format document generation

docloom project — 2026-07-21

Executive Summary

docloom is a Python library and document-generation engine that transforms large-language-model output into finished, production-quality documents across six file formats: PowerPoint, Word, Excel, PDF, HTML, and Markdown. At its core is a schema-validated intermediate representation (IR)—a Pydantic model shaped specifically to survive the constraints of modern structured-output APIs—coupled with deterministic renderers that guarantee reproducibility and quality.

The system comprises two connected layers: the engine (docloom, a Python library, v0.2.0) and the studio (docloom-studio, a free, local-first AI document studio). The engine consumes a Document JSON and renders it to any of six formats through deterministic algorithms with no randomness, no external layout engines, and no magic. The studio is a complete NotebookLM+Gamma-like application built on top —add sources, chat with cited answers, and generate presentations, documents, spreadsheets, diagrams, infographics, and podcast audio, all runnable fully offline against a local model and no third-party account.

This document describes the engine's design, the studio's architecture, the rendering pipeline's quality guarantees, and the system's philosophy of validated, deterministic, local-first document generation.

1. The Core Idea: Validated IR and Deterministic Rendering

The foundational insight is simple but consequential: an LLM never generates code or executable output directly. Instead, it produces a schema-validated JSON document—a Pydantic intermediate representation —that describes *what* a document should contain. A separate set of deterministic renderers then consume that IR and produce real files. This separation of concerns unlocks three capabilities: (1) the LLM output can be validated for correctness before any file is written; (2) a linter can check layout, citations, and contrast quality and feed findings back to the model for self-correction; and (3) the same IR can be rendered to six different formats deterministically without the model knowing or caring which format will be used.

The IR is deliberately shaped to survive real-world constraints. Anthropic structured outputs reject recursive schemas and `oneOf` union definitions. OpenAI's strict mode also forbids `oneOf`. Most structured-output systems struggle with deeply nested types. docloom's IR is therefore deliberately non-recursive: it uses plain tagged unions (a `Literal` type tag plus a `Union`), lists carry a flat `level` integer for indentation rather than nested children, and every schema is flattened so the deepest nesting is shallow. This is not a stylistic choice—it is an architectural constraint imposed by the actual capabilities of the LLM APIs that consume it.

After the model produces a Document JSON, it runs through three mandatory stages: (1) **lenient parsing**, which handles messy output from local or early-stage models; (2) **linting**, which runs machine-readable checks on layout, content, and quality; and (3) **optional feedback**, where findings can loop back to the model for self-correction. Only after all error-severity findings are resolved does the document proceed to rendering.

2. The Document Model and Fourteen Block Types

A docloom Document is a single root object carrying three optional bodies plus metadata. The metadata includes title, subtitle, authors, date, a logo, and a sources list. Each body serves a different output format: `blocks` → reports (DOCX/PDF/HTML/MD), `slides` → decks (PPTX), and `sheets` → workbooks (XLSX). A single Document can carry any mix of these; an application can generate a report and a presentation from the same IR by including both `blocks` and `slides` in the same JSON.

The IR defines exactly 14 block types, each serialized as a tagged union:

- **heading** — numbered section heading (level 1-6)
- **paragraph** — prose body text (plain string or styled spans)
- **bullets** (BulletList) — unordered list with optional nesting via flat level integers
- **numbered** (NumberedList) — ordered list, same nesting model
- **quote** — a block quote
- **code** — a code block (language, title, source-code string)
- **table** — rows × columns with optional header row and type-aware alignment
- **image** — a reference to an image file (plus alt text and caption)
- **callout** — a highlighted information box (style: info/success/warning/danger)
- **divider** — a horizontal separator
- **chart** — columnar data (bar/column/line/area/pie/scatter; PPTX renders native editable charts)
- **stats** (StatRow) — key metrics (label, display-value, optional delta)
- **artifact** — reference to an externally-managed diagram or infographic
- **diagram** — coordinate-free architecture/flow diagram (nodes, edges, groups, direction)

RichText and Citations. Every text field that carries prose can be either a plain string or a list of styled spans. Each span carries text plus optional rich-text markup (bold, italic, code, link) and an optional citation. Citations reference a source by id, and helper methods (`cited_ids()`, `source_numbers()`) produce stable, deterministic citation numbering suitable for displaying as superscript numbers in the rendered output.

SafeStr validation. At the IR boundary, an `AfterValidator` strips C0 control characters, lone UTF-16 surrogates, and the byte-order-mark characters U+FFFE and U+FFFF. These characters are forbidden in OOXML/XML and would crash a renderer or corrupt the output file. By removing them at parse time, they can never reach a renderer.

3. The Studio: A Local-First AI Document Application

docloom-studio is a free, complete document-generation application built on top of the engine. It is "NotebookLM crossed with Gamma"—a user creates a notebook, adds sources (files, URLs, pasted text, or web research), chats with grounded answers, and generates six kinds of artifacts: presentations, documents, spreadsheets, D2 diagrams, infographics, and podcast audio. Everything runs locally with no API key required; a local account is registered on first visit and workspaces are local-only by default.

The user flow: Create a notebook, add sources. Sources can be uploaded files (PDF, DOCX, PPTX, XLSX, CSV, HTML, EPUB, text, Markdown, reStructuredText, JSON, or logs) up to 50 MB each; web URLs; pasted text; or the result of free web research via an integrated agent that plans searches and keeps pages as cited sources. Sources are ingested via lightweight parsers (pdfplumber, python-docx, python-pptx, openpyxl, ebooklib, trafilatura, stdlib CSV) and chunked at roughly 1000 characters with 150-character overlap. Ingestion sanitizes control characters, bidi marks, and zero-width characters to prevent layout attacks.

Grounded chat and retrieval. After ingestion, sources are embedded (default Ollama `nomic-embed-text`) and stored as `.npy` arrays. Chat retrieves k=12 evidence chunks via hybrid retrieval: dense cosine similarity fused with BM25 lexical rank via Reciprocal Rank Fusion (k=60), followed by near-

duplicate dedup and depth-then-breadth per-source coverage. Every answer cites its sources; chat history is persisted and recent turns are folded into retrieval queries to maintain context across multi-turn conversations.

Six artifact kinds. The studio generates six kinds of artifacts: (1) **decks** (PPTX presentations), (2) **documents** (DOCX/PDF/HTML/MD via the docloom engine), (3) **spreadsheets** (XLSX), (4) **D2 diagrams** (rendered client-side in the browser via WASM, never touching docloom's IR), (5) **infographics** (mapped to one of four AntV templates: list, steps, pyramid, grid), and (6) **podcast audio** (a two-host script with optional Kokoro TTS synthesis). Only the first three export through docloom's renderers. Diagrams and infographics render client-side and embed as Artifact blocks when needed in a deck or document.

Five preset guides. The studio includes five one-click generation templates: Study Guide, Briefing, FAQ, and Timeline (all produce documents) and Mind Map (produces a D2 diagram). These templates are hard-coded grounded generations that minimize decisions for new users.

Brand kit application. Every artifact is generated with a brand kit (logo, accent color, fonts) applied. The same brand is stamped on every slide or document, ensuring visual consistency across the entire workspace.

4. Generation Pipelines and Outline-Then-Detail Strategy

The studio uses a **outline-then-per-unit** strategy for generating decks, documents, and spreadsheets. First, a single LLM call produces a high-level outline (slide titles for a deck, section headings for a document, sheet names for a workbook). Then, for each unit, a small-schema LLM call generates that unit's content, with independent retries, progress events, and targeted retrieval using just that unit's topic.

This strategy has two benefits: (1) it lets the system recover gracefully from flaky LLM calls—a single failed unit becomes a skeleton or placeholder rather than sinking the entire artifact; and (2) it reduces context size and token usage by generating focused content for each unit rather than asking for everything at once.

After each schema-shaped generation, the output goes through **`generate_validated()`**: (1) the complete LLM response is attempted; (2) if parsing fails, a lenient parser tries to extract a valid JSON candidate; (3) if a valid parse exists, the linter checks it for error-severity findings; (4) if errors remain, those findings are fed back to the model and the unit is regenerated with escalated temperature; this repeats up to 3 rounds total. A **deterministic citation gate** then strips any citation the model invented that does not reference a real source id, so hallucinated references never ship.

The three other pipelines: (1) **Diagrams** generate D2 source text (not docloom's coordinate-free Diagram IR), lint-validate that it looks like D2 (rejecting Mermaid patterns), and save the source; the browser compiles and renders D2 to SVG/PNG via WASM. (2) **Infographics** emit an `InfographicSpec` (style, title, 3–6 items) mapped to one of four AntV templates, with text deterministically clamped to fixed card limits. (3) **Podcasts** generate a two-host script (host A, guest B) with optional Kokoro TTS synthesis producing a WAV; the transcript ships even if TTS is missing.

An optional **AI slide-image generation** feature ("Nano Banana", via Gemini `gemini-2.5-flash-image`) fills empty hero/image slots; it is controlled by a separate on/off gate, defaults off, and a failure leaves the slot empty rather than sinking the deck.

5. Rendering: Six Formats and One Contract

The engine renders a Document to six formats via a `FORMATS` dispatch table. Every renderer module exposes the same contract: `render(doc, theme, out_path) -> Path`. Given the same IR and theme, rendering is deterministic and reproducible—there is no randomness, no order-sensitive iteration, no file-system-dependent behavior.

Format	Module	Output	Characteristics
pptx	render/pptx.py	PowerPoint	Native editable charts, shapes, and connectors; autofit text; geometric QA audit
docx	render/docx.py	Word	Fixed-layout tables with water-fill column widths, repeating headers; measured autofit
xlsx	render/xlsx.py	Excel	Type-aware cells, formulas, number formats, conditional ranges
pdf	render/typst.py	PDF (in-process)	Compiled from Typst source via the Typst wheel; no external LaTeX or headless browser
html	render/html.py	HTML	Semantic tags, inline SVG diagrams, WCAG 2 landmarks
md	render/markdown.py	Markdown	GitHub-flavored Markdown with fenced code, tables, and reference-style links
typ	render/typst.py	Typst source	Typst markup language; PDF is compiled from this, but source is also available

The six render formats, their modules, and rendering approach.

PDF and `.typ` both come from the **Typst module**. The Typst wheel is compiled into the Python package and used in-process to compile PDF without any external binary, LaTeX installation, or headless browser. `to_typst(doc, theme)` produces the `.typ` source, which can be written directly or compiled to PDF.

6. Quality Guarantees: Autofit, Contrast, and Geometric Auditing

docloom invests in quality beyond simply producing files. Renderers implement measured autofit, WCAG contrast checking, table hardening, and optional post-render geometric auditing.

Measured autofit (PPTX). Python-pptx's built-in `normAutofit` is only recomputed when desktop PowerPoint opens the file—it does not guarantee correct fitting at generation time. docloom's `render/textfit.py` measures real wrapped-text extents against the actual font files using Pillow's `FreeTypeFont`, resolving font families through `pptx.text.fonts.FontFiles` at 200pt. It then bakes the fitted size into every text run. This measurement is necessary because python-pptx has no knowledge of the final rendering environment. Every failure path (missing Pillow, unresolvable font family, corrupt font file) degrades gracefully to a deliberately small size estimate and never crashes.

WCAG 2.x contrast checking. A `contrast_ratio()` function computes the standard luminance formula: (L1 + 0.05) / (L2 + 0.05) using the official 0.2126 / 0.7152 / 0.0722 weights. A linter rule `qa/low-contrast` flags any text run below 4.5:1 (WCAG 2 AA normal text). APCA was deliberately not used; WCAG 2 is the standard implemented here.

DOCX table hardening. Word tables are rendered with fixed layout (`autofit=False`) using content-weighted column widths that are capped at a maximum, renormalized to the frame width, and water-fill-pinned to a minimum. The width is written on every cell (not just the column header) so Word honors it. Header rows repeat (`w:tblHeader`) and body rows use `w:cantSplit` to prevent splits within a row. If no layout can honor the floor, it falls back to Word's native autofit rather than crashing.

Geometric QA audit (reference-free). An optional post-render audit (`render/qa.py`) inspects a built python-pptx Presentation for: off-slide shapes, overlap (with containment filtering), font-family sprawl, palette sprawl, and low contrast. All QA findings are severity `warning` by design, so they never hard-block a deck. This module is imported only by tests, never by the shipped render path.

Optional SVG rasterization. An optional `[diagrams]` extra installs **resvg** (via `resvg-py`) behind `render/raster.py`, so charts and SVG diagrams can embed as real pictures in PPTX and DOCX. Without it, everything still renders: charts fall back to a titled data table and SVG diagrams show as placeholders. This is a graceful degradation—the presence or absence of Pillow and resvg never crashes the system.

7. The Linter: Rules, Severity, and Feedback Loops

`lint(doc, theme)` returns machine-readable findings, each with a rule, severity (error/warning/info), a location (which block), and a message. Only `error` severity blocks rendering (`has_errors()` returns true when any error exists). Warnings signal quality issues but never prevent export; info is purely advisory.

The linter implements roughly 20 rules covering content and layout:

- **Deck overflow.** Both character-budget (cumulative slide text) and physical-height budget (fixed block geometry mirrors from pptx.py as drift-guarded literals)
- **Slide content.** Bullet count (max 7/slide), bullet length (max 130 chars), title length (max 60 chars), empty slides, block variety per slide
- **Typography.** Title strength (no weak or verbless titles), heading-level skips, heading/section balance
- **Tables.** Oversized tables (too many columns or rows)
- **Citations.** Missing sources, invented source ids, uncited factual claims (when evidence is present)
- **Placeholder text.** Detection of unfinished content (e.g., "[your text here]")
- **Images.** File existence, missing alt text
- **Sheets.** Empty cells with formulas, unreferenced ranges
- **Contrast.** WCAG 2 AA compliance (4.5:1 for normal text)
- **Diagrams.** 10 diagram-specific rules (node count, cycle detection, label length, connection validity)

The linter is deliberately **import-light**: its height-budget constants mirror those in `pptx.py` as duplicated literals rather than imports. A drift-guard test in `test_reaudit_lint.py` imports the real constants and asserts equality, so mirrored copies cannot silently diverge. The linter does import `estimate_depth()` from `render/diagram_svg.py` as the single source of truth for the painter's layering algorithm.

8. Diagrams: Two Systems and One Open Decision

Diagrams in docloom are actually **two unconnected systems**. This is not a bug—it is an honest, documented architectural state.

Studio: the D2 editor. In the studio app, users edit diagrams by typing D2 (d2lang) source into a plain `<textarea>` in the left pane. There is no node/edge GUI, no drag-and-drop, no WYSIWYG canvas. The preview compiles D2 to SVG **entirely client-side** via D2's WASM/ELK layout engine (offline, in the browser) and shows it live in the right pane. Every keystroke re-renders with a debounced 700 ms save. The studio applies its brand palette deterministically as a D2 "theme preamble" prepended to the model's source (model owns structure, renderer owns look). A diagram that fails to compile is saved anyway (render errors are preview-only concerns); a "Fix with AI" button POSTs the source and error to a `/repair` endpoint. The PNG is produced by rasterizing the SVG in the browser via `<canvas>` / `Image`, then both SVG and base64 PNG are POSTed back to the server.

Engine: the coordinate-free Diagram IR. The core docloom engine has a **separate** architecture-diagram system. `Diagram` / `DiagramNode` / `DiagramEdge` / `DiagramGroup` carry only structure—no `x`/`y`/`pos`/`size`/`route` fields ever. There are exactly **seven node types** (borrowed from archify): `service`, `client`, `store`, `queue`, `security`, `cloud`, `external`. `DiagramGroup` geometry is derived (bounding box of members plus padding), never authored.

One solver, three emitters. `diagram_svg.solve()` runs layout exactly once, producing a `SolvedDiagram`, then the SVG emitter (`paint_svg`), the native-PPTX emitter (`diagram_pptx`), and the `.drawio` emitter (`drawio.py`) all consume that same solved geometry. The solver is **pure stdlib** Python:

Sugiyama-style layout with rank assignment, proper-graph dummy nodes, barycenter crossing-minimization, median coordinate straightening, orthogonal routing, and grid-packing bands. No external layout engine, no DOM, no browser.

Editability is two-tier. Tier 1 is the Diagram block in the Document JSON (source of truth). Tier 2 is everything derived (SVG, PNG, native PPTX shapes, `.drawio`), regenerated on every render. Nothing today reconciles Tier-2 edits back into Tier 1.

Legibility guarantees. Node labels follow a "nothing authored is lost" rule: `fit_label()` wraps the label and steps the font down (14.5 ⟶ 11.0 pt, then up to three lines at 11/10.5 pt) and never ellipsizes or drops a word. On the PPTX side an 8 pt node-label floor (`MIN_LABEL_PT = 8.0`) is enforced. If a diagram cannot reach 8 pt even after stepping through a three-rung detail ladder (full ⟶ label+sub ⟶ label), it falls back to a raster image at the sparsest layout and emits a warning.

> **The unification is the open decision.** The studio D2 system and the core Diagram IR share no code. The studio editor has never been re-pointed at the Diagram IR. This is the one still-open architectural choice; the design documents record it explicitly. As a consequence, the studio preview and any IR-exported deck can drift.

9. Theme, Contrast, and Font Management

A `Theme` is a bag of **8 semantic color tokens** plus optional local font-file paths: `primary`, `accent`, `background`, `surface`, `text`, `muted`, `font_heading`, `font_body`. Hex colors are validated to `#RRGGBB`. Renderers map these tokens to each format's native color mechanism (OOXML `<a:srgbClr>`, CSS, SVG stroke attributes) and must never hard-code literal colors. This allows a single Document to be re-themed by swapping the Theme object without touching the IR.

Font paths are resolved via Pillow's `FreeTypeFont.truetype()` and cached to avoid repeated disk access. An unresolvable font family degrades to a serif/sans fallback and never crashes. The public Theme includes a `DEFAULT` theme with sensible defaults and a WCAG 2 AA-compliant color palette.

10. Storage, Auth, and Multi-Tenancy

The studio is multi-tenant and storage-agnostic. The default database is **SQLite** at `data_dir()/studio.db` (WAL mode, `foreign_keys ON`). Postgres is supported via `DOCLOOM_DB_URL=postgres://...`; the same `?`-placeholder SQL and a single `MIGRATIONS` list drive both through a translation layer. The schema has **8 migrations** and core tables include `notebooks`, `sources`, `artifacts`, `artifact_versions`, `jobs`, `assets`, `brand_kits`, `settings`, `users`, `workspaces`, `auth_sessions`, `user_settings`, `chat_messages`, and others.

Authentication. Email + password. Passwords are hashed with stdlib **scrypt** ($n=2^{14}$, $r=8$, $p=1$, dklen=32) and stored as `scrypt$salt$hash`; verification uses `hmac.compare_digest`. Sessions are opaque `secrets.token_urlsafe(32)` tokens; only their SHA-256 is persisted in `auth_sessions`, so a database leak cannot be replayed. The `ds_session` cookie is httponly, `samesite=lax`, with a 30-day TTL. On first visit, the user registers a **local** account; everything they create lives in a workspace scoped to that login.

Multi-tenancy. The tenancy model is **users ⟶ workspaces ⟶ notebooks ⟶ sources/artifacts**. Authorization walks the ownership chain and returns **404, not 403**, on cross-tenant access, so IDs cannot be used to probe another tenant's existence. Artifacts are versioned; `save_artifact()` bumps the head version, snapshots into `artifact_versions`, and flips status to `ready` in a single transaction via `UPDATE ... RETURNING`. Revert re-saves an old snapshot as a new version.

Background jobs. Generation runs as in-process async jobs with SSE event streams. Because tasks live in the starting process, a restart loses them; `reconcile_jobs()` marks every DB-running job failed

at startup. Jobs carry a DB heartbeat column (refreshed every 30 sec), enabling lease-mode reconcile on shared Postgres to distinguish crashed nodes from live siblings. Job body concurrency is bounded (default 4, `DOCLOOM_MAX_CONCURRENT_JOBS`).

Secrets at rest. Secret settings (`api_key` fields, Tavily key, Pexels key) are **Fernet-encrypted** and never sent to the client in cleartext. GET masks them as `__stored__`, and PUT treats the mask as "keep the stored value." The Fernet key comes from `DOCLOOM_SECRET_KEY` or auto-generated at `data_dir()/secret.key` (chmod 600).

11. Ingestion, Embedding, and Retrieval Strategies

The studio ingestion pipeline is designed for robustness and security. File uploads are validated against an allowlist (pdf, docx, pptx, xlsx/xlsm, csv, html/htm, epub, txt, md, rst, json, log) with a **50 MB streaming cap** and filename/path-traversal guards. Parsing uses lightweight dependencies: pdfplumber/pypdf, python-docx (including tables), python-pptx (speaker notes), openpyxl, ebooklib (EPUB), trafilatura (HTML), and the stdlib CSV reader.

After parsing, ingestion sanitizes control characters (C0 except tab/CR/LF), bidi marks, and zero-width characters, then chunks at roughly **1000 characters with 150-character overlap**. Two guards protect the ingest path: (1) **URL SSRF guard**—accepts only http(s), rejects private/loopback/link-local/reserved addresses, re-validates every redirect hop by hand, re-checks peer address to defeat DNS-rebinding TOCTOU; (2) **Zip-bomb guard**—ZIP-container formats are checked for entry count, total inflated size (200 MB), and compression ratio before parsing.

Context modes. Each source has a `context_mode`: `full` (every chunk retrieved), `insights` (LLM-generated 3–5 sentence summary embedded as `summary.npy` instead of chunks), or `excluded` (dropped entirely). Embeddings come from the configured provider (default Ollama `nomic-embed-text`) and are stored as `.npy` arrays per source, batched at 64 to respect input caps.

Hybrid retrieval. Retrieval (`embeddings.py`) fuses dense cosine similarity with pure-Python **BM25** lexical rank via **Reciprocal Rank Fusion** (k=60), followed by near-duplicate dedup and depth-then-breadth per-source coverage floor. It is brute-force—described in code as "instant to ~100k chunks; add hnswlib past that." Sources whose stored vector count or embedding dimension no longer match their chunks (e.g., after switching embedding models) are flagged `status='stale'` so the UI can prompt re-ingest.

12. Running the Studio and Configuration

Launching the studio is a single command from the repository root:

```
# Windows (PowerShell)
studio.ps1

# macOS / Linux / Git Bash
studio.sh
```

The launcher installs dependencies (uv, Node 22+, npm), builds the frontend, and starts a single FastAPI process that serves both API and SPA on one port, then opens a browser. Prerequisites are `uv`, Node 22+, and npm on `PATH`. The launcher exits with an install hint if any is missing.

Launcher flags: `-Rebuild`/`--rebuild` (force fresh web build), `-Setup`/`--setup` (force dependency reinstall), `-Port`/`--port` (port override), `-NoBrowser`/`--no-browser` (sets `DOCLOOM_STUDIO_NO_BROWSER`). On every start, the launcher verifies that resvg is importable and self-installs `resvg-py>=0.3.3` if missing—without it, the studio's browser-rendered diagrams, charts, and infographics can export as silent blanks.

Docker. Build from the repository root with `docloom-studio/Dockerfile` and run `-p 8899:8899` with a `/data` volume.

Host and port. Default to **127.0.0.1:8899** (`http://127.0.0.1:8899`), overridable via `DOCLOOM_STUDIO_HOST` and `DOCLOOM_STUDIO_PORT`. Startup opens a browser unless `DOCLOOM_STUDIO_NO_BROWSER` is set.

Data directory. The studio stores everything—SQLite DB, uploaded sources, assets, exports, cache—under `%LOCALAPPDATA%/docloom-studio` on Windows or `~/docloom-studio` elsewhere, overridable via `DOCLOOM_STUDIO_HOME`.

Providers. The provider layer supports: ollama, llama-server, lmstudio, openai, anthropic, gemini. Each has its own HTTP shape (OpenAI-compatible chat completions, Ollama `/api/chat`, Anthropic `/v1/messages`, Gemini's native API). Generation and embeddings are configured separately inside Settings and the model list is fetched live from the base URL.

In-code defaults: Generation is **Ollama** with model `qwen3.5:9b` and `max_tokens` 16384 (raised from 8192), pointing at `http://localhost:11434`. Embeddings default to Ollama `nomic-embed-text`, same host. Gemini is fully supported; `gemini-2.5-flash` allows 65536 output tokens. Any non-default provider—e.g., running Gemini as the default—is a per-machine stored DB setting, not the shipped code default.

Other settings: Podcast TTS defaults to `kokoro` (local voices), language `a` (American English), host voice `af_heart`, guest voice `am_michael`. Image generation ("Nano Banana", Gemini `gemini-2.5-flash-image`) is a separate cloud/paid surface, disabled by default. Optional research (Tavily) and asset (Pexels) API keys default empty.

13. Architecture and Public Interfaces

The engine exports a minimal public API (**package version 0.2.0**). `docloom/__init__.py` re-exports the public surface: `Document` and all block models; `lint`/`has_errors`/`Finding`; `render`/`render_diagram`/`FORMATS`/`RenderError`; `Theme`/`DEFAULT`; `llm_schema`/`parse_llm_output`/`AUTHORING_GUIDE`; `diagram_hash`/`ensure_ids`.

CLI. The `docloom` command-line tool has subcommands: `render` (takes `-f`/formats, `-o`/out, `--theme`, `--no-lint`, `--diagram-sources`; refuses to write on error findings unless `--no-lint` passed; exits code 2 on errors), `lint` (exits 1 on errors), `schema` (outputs JSON Schema), `guide` (outputs authoring guide), `theme` (manages themes).

MCP server. The `docloom-mcp` entry point (stdio transport) exposes three tools: `get_document_schema`, `lint_document`, `render_document`. `render_document` refuses to render on error findings unless `no_lint=True` and resolves output directory to absolute path.

JSON API (studio). The studio's FastAPI server exposes REST routers: `auth`, `notebooks`, `sources`, `assets`, `artifacts`, plus inline settings/providers/themes/layout endpoints. File serving is confined to ownership-checked, per-artifact routes (older `/api/files?path=` route was removed due to path traversal risks).

14. Design Principles and Philosophy

Several core principles guide docloom's architecture and have resisted the pressure to compromise:

- **Validated IR, not code.** The LLM produces a schema-validated data structure, never executable output. The schema is deliberately shaped (non-recursive, plain tagged unions, no `oneOf`, stripped constraints) to survive real structured-output constraints from Anthropic and OpenAI.
- **Deterministic renderers.** Given an IR and a theme, rendering is reproducible. Diagrams are solved exactly once and every emitter consumes the same geometry; `.drawio` output is byte-deterministic.

- **Lint before render, and let findings self-correct.** A machine-readable linter runs before any file is written. Only `error` severity blocks rendering; warnings stay as signals so they never hard-block a whole document. Findings can loop back to the model for self-correction.
- **Nothing authored is lost, nothing drawn is unreadable.** Node labels are never truncated or word-dropped. When text cannot be made legible at the layout's density, the system steps down detail and warns rather than silently shipping something unreadable.
- **Local-first and multi-tenant-safe.** The studio runs offline against a local model with a local account. Cross-tenant access returns 404; session tokens are stored only as hashes; secrets are encrypted at rest; file serving is confined to ownership-checked, per-artifact routes.
- **Graceful degradation everywhere.** Optional dependencies (Pillow for autofit, resvg for rasterization, Kokoro for TTS) improve output when present and degrade to safe fallback—never a crash—when absent.

15. Limitations, Roadmap, and Future Directions

While docloom is feature-complete for document generation, several decisions remain open or constrain the current scope:

Diagram unification. The studio D2 editor and the core Diagram IR are separate systems. Unifying them—pointing the studio at the Diagram IR instead of D2—would enable node/edge GUI, drag-and-drop WYSIWYG, and live preview without WASM, but would require rebuilding the editor. This is the one still-open architectural decision.

Retrieval scale. Hybrid retrieval with BM25 and RRF is brute-force ("instant to ~100k chunks"). Scaling past that requires adding hnswlib for dense-only approximate nearest-neighbor search, trading some recall for speed. This trade-off has not yet been made.

Diagram-to-Tier-2 reconciliation. Edits made in `.drawio` or PPTX shapes do not reconcile back to the Diagram IR. The IR is Tier 1 (source of truth); derived formats (SVG, PPTX, `.drawio`) are Tier 2 (regenerated on every render). This is intentional but means users cannot edit in PowerPoint and re-import changes.

Optional dependencies. Pillow and resvg are optional. Core installation renders all six formats without either, but measured autofit (PPTX) and SVG rasterization (charts in DOCX/PPTX) are unavailable. This is a graceful degradation but a limitation.

Diagram legibility guarantee. An 8 pt label floor in PPTX can force fallback to a raster image at sparsest layout, which is readable but less polished than vector shapes. Future work could explore higher-density packing or interactive zoom in the studio.

Appendix: Headline Numbers and Summary

14	6	7	~20	0.2.0
Block types	Formats	Diagram node types	Linter rules	Package version

docloom is a production-grade document-generation engine that bridges the gap between LLM output and real, editable files. It enforces validated intermediate representation, deterministic rendering, quality checking before export, and graceful degradation. The studio layer builds a complete local-first AI document application on top, supporting notebooks, retrieval-augmented chat, and six kinds of artifact generation—all offline and free.

The system's core insight is simple: separate the *what* (LLM-authored IR) from the *how* (deterministic renderers). This separation enables validation, lint-driven self-correction, multi-format export, and

reproducibility. Every design decision follows from this principle, and every optional component degrades gracefully rather than failing loud.